

Bloque 1. Introducción al desarrollo de interfaces

TEMA 1

Introducción a las interfaces de usuario

1. Introducción	2
2. Interfaces de usuario	2
2.1. Concepto de interfaz de usuario	3
2.2. Tipos de interfaces de usuario	6
2.2.1. Interfaces de escritorio (Windows, MacOS, Linux)	6
2.2.2. Interfaces móviles (Android, iOS)	7
2.2.3. Interfaces híbridas (apps multiplataforma)	7
2.2.4. Realidad aumentada (AR) y Realidad Virtual (VR)	7
2.2.5. Interfaces gestuales (Kinect, sensores)	8
2.2.6. Interfaces por voz (asistentes personales: Siri, Alexa,...)	8
2.3. Elementos de las GUI. Propiedades y características.	9
2.4. Herramientas de edición de interfaces	12
3. Introducción al diseño de interfaces	14
3.1. Interacción persona-ordenador	14
3.2. Diseño de una interfaz. El diseño gráfico	15
3.3. Prototipos. Elementos clave de un prototipo	19
3.4. Planteamiento y diseño de una interfaz	21
3.4.1. Área de redacción	22
3.4.2. Área de producción	22
3.4.3. Área técnica	22
3.4.4. Área artística	22
3.5. El color	23
3.5.1. Sistema RGB	23
3.5.2. Matiz, saturación y brillo	24
3.5.3. Psicología del color	25
3.6. Herramientas de diseño y prototipado (sin código)	25

1. Introducción

Uno de los campos de la informática que se encuentra en constante evolución y experimentación es el del diseño y desarrollo de interfaces de usuario. Y esto no sólo es debido a la evolución de herramientas de desarrollo o aparición de nuevos estándares, sino también el desarrollo de interfaces hardware que dan la vuelta a la forma de interacción habitual con las máquinas.

En este primer tema haremos una introducción a las interfaces de usuario y a los elementos que habitualmente se utilizan dentro de una interfaz para conseguir una buena interacción hombre-máquina.

También haremos un repaso a algunas de las herramientas que podemos emplear para desarrollar los diferentes tipos de interfaz, tanto de software libre como de privativo. Actualmente hay multitud de ejemplos de aplicaciones informáticas con las interfaces de usuario correspondientes, que ayudan en la vida cotidiana de los seres humanos.

Las interfaces de usuario están en continua evolución y se han de conocer una serie de herramientas que permitan su desarrollo.

2. Interfaces de usuario

Comenzaremos recordando el funcionamiento de un sistema informático. ¿Cómo consigue un usuario manejar como a él le interesa un ordenador? El conjunto de piezas que componen el hardware de un ordenador deberá llegar a funcionar según los deseos del usuario. Pero la relación entre uno y otro tendrá algunos actores intermedios, como se puede ver en la figura 1.

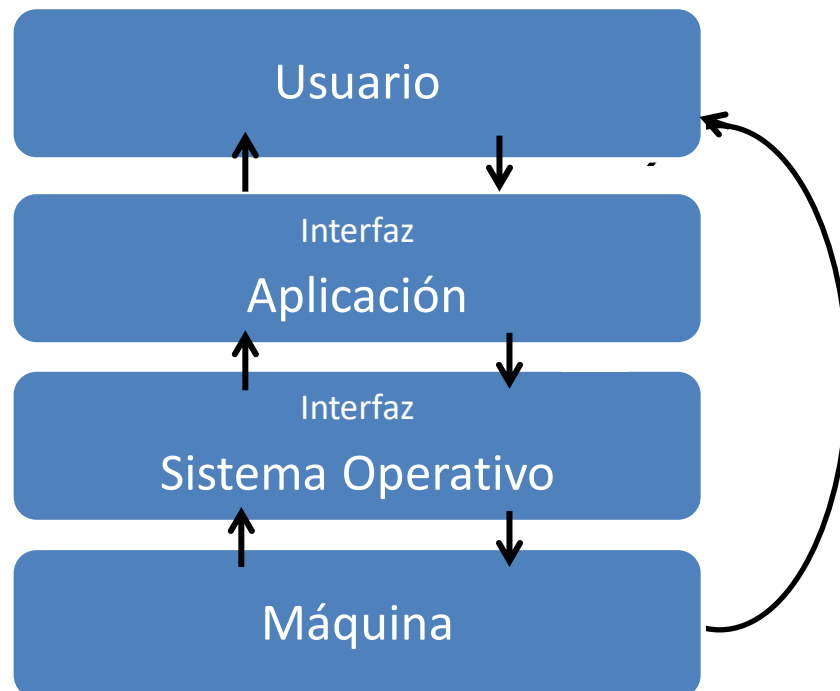


Fig. 1. Actores en la relación con la máquina

En primer lugar se encuentra el sistema operativo, encargado de la gestión y la coordinación de las tareas que llevan a cabo un intercambio de información entre los diferentes recursos. Se compondrá de un sistema de entrada/salida, de la gestión de procesos, de la gestión de la memoria principal o del sistema de archivos, entre otros. A la vez actuará de interfaz entre el hardware y las aplicaciones utilizadas.

En segundo lugar tendremos las aplicaciones. Éstas se encuentran por encima del sistema operativo (el cuál necesitarán, adaptándose a él) y por debajo de los usuarios. Las aplicaciones tienen unas funcionalidades concretas y ayudan a los usuarios a conseguir sus objetivos.

¿Dónde se encuentran las interfaces? Las interfaces forman parte de las aplicaciones. Son la parte de las aplicaciones con la que se relacionarán los usuarios.

Hay muchos tipos de aplicaciones informáticas. Se pueden agrupar según la finalidad, el entorno de uso, si son de uso genérico o si son hechas a medida, pero también se pueden agrupar según sean aplicaciones de gestión empresarial, para el ocio, ofimáticas, de gestión del hardware, aplicaciones en un entorno cliente-servidor, en un entorno web...

2.1. Concepto de interfaz de usuario

Hay muchos tipos de interfaces de usuario en muchos ámbitos diferentes. Por esta razón es difícil encontrar una definición única que deje claro el concepto.

Una aplicación informática tendrá varias interfaces de usuario. Una **interfaz de usuario** es un conjunto de elementos (que pueden pertenecer al software o al hardware) que ofrecen una información al usuario, y permiten, además la interacción (física o lógica) entre el usuario y el ordenador, por medio de un dispositivo periférico o un enlace de comunicaciones.

Dentro de las interfaces de usuario diseñadas para aplicaciones informáticas, nosotros nos vamos a fijar en las interfaces de usuario gráficas o GUI (graphical user interface). Las GUI permitirán al usuario interactuar con el sistema informático de más maneras que la que históricamente ha sido la única alternativa hasta hace unos años: teclear instrucciones complejas y difíciles de entender para el usuario no experto.

Una **interfaz gráfica de usuario (GUI - Graphical User Interface)** es el espacio visual y funcional donde el usuario interactúa con un sistema informático, una aplicación o un dispositivo. Está formada por elementos visuales como ventanas, botones, menús, iconos, formularios, textos y otros componentes gráficos que permiten que el usuario controle y reciba información del sistema de manera sencilla, intuitiva y eficiente.

Actualmente podemos encontrar un uso generalizado de las interfaces gráficas de usuario. La gran mayoría de los sistemas operativos tienen este tipo de interfaz, de tal manera que integrar una aplicación en el sistema operativo sea muy sencillo, y sólo haya que adaptar algunos parámetros. Incluso hay dispositivos que han desarrollado su sistema operativo propio, como los dispositivos móviles, con sus interfaces propias. En la figura 2 se puede ver un ejemplo con el iPad, de Apple.



Fig. 2. Interfaz gráfica iPad

Las características clave de las interfaces de usuario son las siguientes:

- Visual y táctil (en pantallas táctiles).
- Basada en metáforas visuales reconocibles (carpetas, papeles, botones)
- Interactiva: responde a las acciones del usuario (clics, toques, gestos)
- Puede estar en dispositivos diversos: ordenadores, móviles, tablets, dispositivos inteligentes

Por otra parte, hemos de hablar también del papel del diseñador de interfaces en proyectos digitales

El **diseñador de interfaces gráficas** es el profesional encargado de crear estos espacios visuales donde los usuarios interactúan con la tecnología. Su trabajo permite que la experiencia sea:

- **Fácil de entender:** Que el usuario sepa qué hacer sin explicaciones complicadas.
- **Eficiente:** Que pueda realizar sus tareas con el mínimo esfuerzo y tiempo.
- **Atractiva:** Que el diseño sea visualmente agradable y coherente.
- **Accesible:** Que pueda usarla cualquier persona, independientemente de sus capacidades.

Las **principales funciones** que desempeña un diseñador de interfaces gráficas son:

- Investigar y comprender las necesidades y comportamientos de los usuarios.
- Diseñar estructuras, componentes y flujos de navegación.
- Elegir colores, tipografías, iconos y estilos que transmitan la identidad del producto y faciliten su uso.
- Elaborar prototipos, wireframes y mockups para mostrar la interfaz antes de desarrollarla.
- Colaborar con desarrolladores para asegurar que el diseño se implemente correctamente.
- Realizar pruebas y mejoras continuas basadas en el feedback de usuarios.

Además, los diseñadores gráficos deben **colaborar con equipos de desarrollo, UX y cliente** ya que el diseño de interfaces es un proceso colaborativo que involucra varios perfiles profesionales:

- **Diseñadores UX (Experiencia de Usuario):** Se centran en la investigación del usuario, análisis de necesidades, arquitectura de la información y la usabilidad general.
- **Diseñadores UI (Interfaz de Usuario):** Se encargan del aspecto visual y la interacción directa, es decir, el diseño gráfico y la planificación de componentes.
- **Desarrolladores front-end:** Programan la interfaz en código, implementando los diseños con tecnologías web, móviles o de escritorio.
- **Clientes y usuarios finales:** Proveen los requisitos, expectativas y validan el producto para que cumpla con sus objetivos.

El diseñador de interfaces actúa como puente entre estas partes, traduciendo las necesidades del usuario y cliente en un diseño visual y funcional que los desarrolladores puedan construir.

En cuanto a los **objetivos fundamentales** que una interfaz gráfica eficaz debe cumplir se citan a continuación:

- **Facilitar la interacción**
 - Permitir que el usuario realice sus tareas sin confusión ni esfuerzo excesivo.
 - Ofrecer una navegación clara y componentes reconocibles.
 - Proveer feedback inmediato a las acciones del usuario para que se sienta en control.

- **Transmitir información**

- Mostrar datos, estados o mensajes de forma comprensible y visualmente organizada.
- Priorizar la información importante para evitar saturar al usuario.
- Usar códigos visuales (colores, iconos, tipografía) que ayuden a interpretar el contenido.

- **Generar confianza**

- Que el usuario perciba la interfaz como profesional, fiable y segura.
- Mantener coherencia visual que refuerce la identidad del producto o marca.
- Evitar errores y ofrecer ayudas o mensajes claros cuando algo no funcione.

2.2. Tipos de interfaces de usuario

Las interfaces gráficas de usuario pueden clasificarse según el tipo de dispositivo o la tecnología con la que se interactúa. Cada tipo tiene características propias, retos específicos y usos comunes. Es importante conocer estas diferencias para planificar y diseñar interfaces adecuadas y efectivas.

2.2.1. Interfaces de escritorio (Windows, MacOS, Linux)

Estas interfaces son las más clásicas y tradicionales, diseñadas para ordenadores personales y estaciones de trabajo.

Como **características principales** podemos citar:

- Uso con teclado y ratón.
- Ventanas múltiples, menús desplegables, barras de herramientas.
- Suelen permitir gran cantidad de información y funcionalidades en pantalla.
- Espacios amplios para elementos visuales.

Los **sistemas operativos** comunes en los que se utilizan son:

- **Windows:** Dominante en entornos empresariales y domésticos.
- **MacOS:** Usado en equipos Apple, reconocido por su diseño cuidado.
- **Linux:** Popular en entornos técnicos y desarrolladores, con interfaces variadas (GNOME, KDE, etc.).

2.2.2. Interfaces móviles (Android, iOS)

Estas interfaces están diseñadas para dispositivos con pantalla táctil, como smartphones y tablets.

Sus características principales son:

- Interacción táctil: toques, deslizamientos, pellizcos.
- Espacios limitados, necesidad de simplificar la interfaz.
- Uso frecuente en movimiento y con una sola mano.
- Interfaces orientadas a gestos, accesos rápidos y notificaciones.

Los **sistemas operativos** principales en los que se utilizan son:

- **Android**: Sistema abierto y altamente personalizable.
- **iOS**: Sistema cerrado, enfocado en experiencia uniforme y controlada.

2.2.3. Interfaces híbridas (apps multiplataforma)

Son aplicaciones que funcionan en varios tipos de dispositivos con una sola base de código.

Entre sus características podemos citar que:

- Se adaptan automáticamente a pantallas grandes y pequeñas.
- Permiten un desarrollo más rápido y económico.
- Usan tecnologías como React Native, Flutter, Ionic, Quasar, Kotlin Multiplatform.
- Desafío: lograr buen rendimiento y experiencia nativa en cada plataforma.

2.2.4. Realidad aumentada (AR) y Realidad Virtual (VR)

Se trata de interfaces que integran o crean entornos digitales para aumentar o sustituir la realidad.

Las interfaces de Realidad Aumentada (AR) superponen información digital sobre la imagen del mundo real. Algunos ejemplos: juegos como Pokémon Go y aplicaciones de navegación o diseño.

Las interfaces de Realidad Virtual (VR) sumergen al usuario en un entorno completamente digital, usan gafas o cascos especiales y se usan en aplicaciones de videojuegos, formación o simulación.

Como características comunes podemos destacar que ambas son Interfaces 3D, inmersivas y requieren sensores de movimiento, cámaras o mandos.

Los desafíos que deben superar son evitar mareos y diseñar interacciones intuitivas en 3D.

2.2.5. Interfaces gestuales (Kinect, sensores)

Las interfaces gestuales permiten controlar dispositivos mediante movimientos del cuerpo o gestos con las manos.

Algunos ejemplos son:

- **Kinect:** sensor de Microsoft para videoconsolas que detecta movimientos.
- Cámaras y sensores en dispositivos para reconocer gestos.
- Aplicaciones en juegos, presentaciones, control sin contacto.

Como principales características, podemos destacar que:

- Eliminan la necesidad de dispositivos físicos (ratón, teclado).
- Producen una interacción natural, pero pueden ser menos precisas.
- Requieren diseño claro de gestos para evitar confusión.

2.2.6. Interfaces por voz (asistentes personales: Siri, Alexa,...)

En las interfaces por voz la interacción tiene lugar mediante comandos y respuestas habladas.

Como ejemplos podemos citar **Siri (Apple)**, **Alexa (Amazon)**, **Google Assistant**, que se usan en smartphones, altavoces inteligentes, coches y más

Entre sus principales características se encuentran que:

- Permiten controlar dispositivos sin usar manos.
- Son ideales para multitarea o accesibilidad.

Por otra parte, plantean los siguientes desafíos: reconocimiento de voz preciso, interpretación del lenguaje natural de la forma más precisa posible y dar respuesta contextual.

Se recoge un resumen de los distintos tipos de interfaces con sus características:

Tipo de interfaz	Dispositivo típico	Interacción principal	Retos principales
Escritorio	PC, portátil	Ratón y teclado	Complejidad, multitarea
Móvil	Smartphone, tablet	Táctil	Espacio reducido, gestos
Híbrida	Móviles y escritorio	Táctil y ratón	Adaptación multi-dispositivo
AR / VR	Gafas, smartphones	Movimiento, visual 3D	Inmersión, ergonomía, mareos
Gestual	Sensores, cámaras	Movimientos corporales	Precisión, aprendizaje de gestos
Voz	Altavoces, smartphones	Comandos de voz	Reconocimiento, contexto, ruido

2.3. Elementos de las GUI. Propiedades y características.

Estudiada la evolución de las interfaces gráficas de usuario a lo largo del tiempo, podemos observar como el denominador común ha sido llegar a ofrecer a los usuarios unas interfaces con una usabilidad óptima. Para conseguir esto se ha establecido que las interfaces deben cumplir una serie de características que, entre otras, serán:

- **Accesible e intuitiva:** Una interfaz debe ser intuitiva en su uso, debe mostrar con claridad las funcionalidades que ofrece y facilitar llegar a las mismas de manera sencilla y clara.
- **Uso de metáforas:** Para identificar las funcionalidades que representan o los objetivos que simularán, las interfaces deben utilizar metáforas que vinculen sencillamente icono o imagen con funcionalidad u objetivo.
- **Aprendizaje y uso fácil:** Las interfaces deben ser fáciles de usar y también de aprender por parte de los usuarios principiantes.
- **Consistencia:** Las interfaces deben seguir un mismo diseño y estructura entre ellas y también con otras interfaces de aplicaciones análogas. También deberán ser consistentes cuando se ejecuten en diferentes entornos.

- **Ofrecer el control de las interfaces:** Una interfaz debe saber entregar su control al usuario que la utilizará.
- **Anticipación:** Se deberán prever los posibles errores que pueda cometer un usuario o las necesidades que pueda demostrar y ofrecer soluciones antes de que aparezcan o controlándolos y ofreciendo soluciones.
- **Legibilidad:** Deberán ser fácilmente interpretables y ofrecer una legibilidad adecuada a los usuarios.
- **Autonomía:** Un usuario no debe necesitar más información o ayuda que la que una interfaz le ofrece o, la que puede encontrar a partir de las indicaciones que le mostrará esta interfaz.
- **Reducir carga de memoria:** Para poder utilizar una interfaz más de una vez no habrá de obligar a los usuarios a recordar la ubicación de las funcionalidades, sino que éstas deberán ser sencillas de encontrar. Esto hará que no sea indispensable memorizar muchas informaciones para aprender a utilizar una interfaz determinada.
- **Internacionalización de la interfaz:** Debe permitir ser entendida y utilizable por usuarios de diferentes culturas e idiomas o bien haciéndola internacional con iconos fácilmente reconocibles o con la posibilidad de seleccionar el idioma con el que se querrá trabajar.
- **Valores iniciales:** También conocidos como valores por defecto o estándar. Son los valores que en las interfaces que llevan un formulario incorporado aparecerán seleccionados inicialmente. Además, deben poder ser descartados de manera sencilla.
- **Ley de Fitts:** Una interfaz debe conseguir optimizar la ley de Fitts. Esta ley, en ergonomía, moldeará el movimiento humano, haciendo una estimación del tiempo que puede necesitar un ser humano para mover un puntero desde una zona de la pantalla hasta otra teniendo en cuenta variables como los objetivos, la distancia hasta alcanzarlos y el tamaño que tendrán.

Las interfaces gráficas de usuario (GUI a partir de ahora) disponen de una serie de elementos propios, comunes a muchas de las GUI desarrolladas, que dispondrán de unas características y propiedades. Con estos elementos se facilitará el desarrollo de las GUI de las aplicaciones informáticas, lo que ofrece la posibilidad de cumplir la mayoría de las características definidas.

Para que el usuario pueda utilizar estos elementos, deberá utilizar alguno de los dispositivos de entrada/salida que se han ido desarrollando a lo largo de los últimos años. El teclado del ordenador ha sido el dispositivo indispensable, como lo es hoy en

día el ratón. Pero hay otros como el *touchpad* (ratón táctil), la tableta gráfica, el *trackball* (bola), los *joysticks*, las pantallas táctiles o los micrófonos. Como dispositivos de salida, además de la pantalla, serán importantes los altavoces o los LED informativos.

Algunos de estos dispositivos serán indispensables para el funcionamiento correcto de algunos de los elementos que ofrecen las GUI. Cada interfaz gráfica utilizará algunos elementos en función de su entorno de trabajo, y no todas tienen los mismos elementos.

Las interfaces gráficas están formadas por diversos elementos visuales y funcionales que permiten la interacción entre el usuario y el sistema. Los componentes más comunes incluyen:

- **Botones:** Elementos clicables que ejecutan acciones específicas. Pueden variar en forma, tamaño, color y estado (activo, inactivo, presionado).
- **Menús:** Listados desplegables o fijos que agrupan opciones o comandos para la navegación o selección.
- **Formularios:** Conjuntos de campos donde el usuario introduce datos (texto, selección, casillas, etc.). Incluyen validaciones y ayudas para facilitar la entrada correcta.
- **Cuadros de diálogo:** Ventanas emergentes que solicitan información, presentan alertas o confirman acciones del usuario.
- **Sliders (deslizadores):** Controles para seleccionar valores dentro de un rango continuo o discreto, como volumen o brillo.

Otros componentes pueden ser pestañas, listas desplegables, barras de progreso, iconos interactivos, etc.

La **organización de la interfaz en ventanas** es clave para gestionar la información y las acciones del usuario. Podemos distinguir dos tipos fundamentales:

- **Ventanas modales:** Son ventanas que requieren la atención inmediata del usuario, bloqueando la interacción con otras partes de la interfaz hasta que se cierre o complete una acción. Ejemplos: diálogos de confirmación, formularios críticos.
- **Ventanas no modales:** Permiten al usuario interactuar simultáneamente con otras ventanas o elementos de la interfaz, facilitando multitarea o consultas paralelas. Ejemplo: ventanas de ayuda o paneles flotantes.

Además, según la organización de las ventanas de la aplicación podemos distinguir dos tipos de interfaces:

- **SDI (Single Document Interface):** Cada ventana de la aplicación maneja un único documento o tarea. Ejemplo: editores de texto donde cada documento está en su propia ventana independiente.
- **MDI (Multiple Document Interface):** La aplicación contiene múltiples documentos o ventanas dentro de una ventana principal, permitiendo organizarlos y cambiar entre ellos fácilmente. Ejemplo: programas de edición gráfica o desarrollo que gestionan varios archivos simultáneamente.

2.4. Herramientas de edición de interfaces

El desarrollo de una interfaz gráfica de usuario dependerá mucho del lenguaje de programación que se utilice y de su entorno de desarrollo.

Será muy diferente (y se necesitarán herramientas de software diferentes) desarrollar una aplicación informática con un lenguaje de programación de **tercera generación**, en el que se trabajará con bibliotecas para poder tener acceso a un entorno gráfico, o trabajar con un lenguaje de **cuarta generación**, con componentes gráficos y de acceso a datos incorporados.¹

Otra diferencia importante será desarrollar una aplicación para un entorno Web o desarrollarla por un entorno de ventanas (Winforms). Los componentes, los lenguajes, la metodología de trabajo y las posibilidades serán diferentes en los dos casos.

Las herramientas de creación y edición de interfaces son un tipo de software que ayudará al programador a desarrollar interfaces que sigan las indicaciones establecidas en la fase de diseño a partir del análisis de las necesidades. Deberán tener en cuenta todas las fases de desarrollo de un proyecto informático.

Este tipo de software deberá tener en cuenta muchas disciplinas, ya que la creación de una interfaz de usuario engloba conceptos de diseño gráfico, de lingüística, de ergonomía, de sociología, de programación y tecnología informática y de psicología cognitiva, entre otros.

Hay muchas herramientas diferentes en el mercado en función del lenguaje de programación que se utilice, en función del entorno de desarrollo o de si la herramienta es propietaria o es libre. Para conseguir desarrollar de manera sencilla las interfaces,

¹ Generaciones de lenguajes. Lenguajes de tercera generación: C, C++, C#, Java, Delphi,... Lenguajes de cuarta generación: Visual Basic .NET, PL-SQL, PHP, ASP,...

las herramientas incorporan diversos elementos preprogramados que facilitan mucho la creación de interfaces gráficas complejas de una manera sencilla. Así será muy sencillo añadir a una interfaz un menú o un icono, tan sencillo como desplazar este elemento al panel principal donde se desarrolla la interfaz, y modificar posteriormente sus propiedades. Estos elementos pueden ser para fomentar la iteración con el usuario (ayudas, actualización de datos, acciones sobre cursores...) o elementos para configurar los (iconos, ventanas, controladores de dispositivos, menús...). Además, se pueden gestionar los posibles errores por parte del usuario en utilizar la interfaz, procesar excepciones, gestionar y controlar sus accesos y gestionar los dispositivos de entrada.

Otras características a tener en cuenta de las herramientas para la creación y edición de interfaces gráficas es que éstas son tipos de herramientas del tipo **WYSIWYG**, es decir, se podrán observar de manera directa los cambios en el diseño y desarrollo de las interfaces. Lo que hacen es lo que verá el usuario final de las aplicaciones, y se pueden ver los resultados de forma inmediata.

A continuación se enumeran algunas de las muchas herramientas de desarrollo de interfaces existentes:

- **Microsoft Visual Studio.** Se trata de un entorno de desarrollo integrado desarrollado por Microsoft para sistemas operativos Windows. Soporta los lenguajes de programación siguientes: Visual C++, Visual C#, Visual J#, ASP.NET y Visual Basic .NET.
- **Eclipse.** Desarrollado inicialmente por IBM. Actualmente desarrollado por una fundación sin ánimo de lucro, lo que hace la herramienta gratuita y de código abierto. Soporta, entre otros, los lenguajes Java, C++, Perl y PHP. Es multiplataforma.
- **Glade.** Permite la creación de interfaces gráficas de usuario. Es muy utilizada en entornos XML. Permite el desarrollo de interfaces gráficas basadas en lenguaje C, C++, C#, Java, Python y otros.
- **NetBeans.** Desarrollado inicialmente por Sun, que actualmente patrocina los proyectos de desarrollo. Herramienta de código abierto y gratuito. Soporta, entre otros, los lenguajes Java, JSP, C++ y PHP. Es multiplataforma.
- **MonoDevelop.** Entorno de desarrollo gratuito y de código abierto. Actualmente patrocinado por Novell. Diseñado para C#, también soporta Java o Boo. Es multiplataforma.

3. Introducción al diseño de interfaces

3.1. Interacción persona-ordenador

Lo primero que debemos plantearnos a la hora de diseñar una interfaz es: *¿qué es la interacción persona-ordenador?* Podemos encontrar multitud de definiciones, pero nos vamos a quedar con que es la **disciplina que estudia el intercambio de información entre las personas y los ordenadores, cuyo objetivo es que este intercambio sea más eficiente, es decir, disminuyan los errores, se incremente la satisfacción, etc.**

Licklider y Clark, en 1962, elaboran una lista con los diez problemas más comunes que deberán ser resueltos para facilitar la interacción entre las personas y los ordenadores:

1. Compartir el tiempo de uso de los ordenadores entre muchos usuarios.
2. Un sistema de entrada-salida para la comunicación mediante datos simbólicos y gráficos.
3. Un sistema interactivo de proceso de las operaciones en tiempo real.
4. Sistemas de almacenamiento masivo de información que permitan su rápida recuperación.
5. Sistemas que faciliten la cooperación entre personas en el diseño y programación de grandes sistemas.
6. Reconocimiento por parte de los ordenadores de la voz, de la escritura manual impresa y de la introducción de datos a partir de escritura manual directa.
7. Comprensión del lenguaje natural, sintáctica y semánticamente.
8. Reconocimiento de la voz de varios usuarios por el ordenador.
9. Descubrimiento, desarrollo y simplificación de una teoría de algoritmos.
10. Programación heurística o a través de principios generales.

Hansen (1971) en su libro *User Engineering Principles for Interactive Systems* hace la primera enumeración de principios para el diseño de sistemas interactivos:

1. Conocer al usuario.
2. Minimizar la memorización, sustituyendo la entrada de datos por la selección de ítems, usando nombres en lugar de números, asegurándose un comportamiento predecible y proveyendo acceso rápido e información práctica del sistema.

3. Optimizar las operaciones mediante la rápida ejecución de operaciones comunes, la consistencia de la interfaz y organizando y reorganizando la estructura de la información basándose en la observación del uso del sistema.
4. Facilitar buenos mensajes de error, crear diseños que eviten los errores más comunes, haciendo posible deshacer acciones realizadas garantizar la integridad del sistema en caso de un fallo de software o hardware.

La **Teoría de la Gestalt** proviene de la psicología y explica cómo percibimos patrones y estructuras visuales. En diseño, se utiliza para crear interfaces que el usuario pueda entender de forma rápida y natural, gracias a la forma en que organizamos los elementos. Sus principios clave son:

- **Proximidad:** Elementos cercanos se perciben como relacionados.
- **Semejanza:** Elementos similares en forma, color o tamaño se agrupan visualmente.
- **Continuidad:** La mente tiende a seguir líneas y patrones continuos.
- **Figura-fondo:** Distinguir un objeto (figura) de su fondo para enfocar la atención.
- **Cierre:** La mente completa formas incompletas para crear figuras completas.
- **Simetría:** Elementos simétricos se perciben como parte de un mismo grupo o como visualmente agradables.

Estos principios ayudan a organizar contenido, menús, botones y otros elementos para mejorar la comprensión y usabilidad.

3.2. Diseño de una interfaz. El diseño gráfico

El diseño gráfico consiste en programar, proyectar y realizar comunicaciones visuales de aplicaciones u otro tipo de herramientas software que generalmente serán transmitidos por medios industriales. A día de hoy, esta área de desarrollo tiene a sus propios profesionales, denominados *diseñadores gráficos*. Se destacan sus tres grandes funciones:

- Función estética.
- Función publicitaria.
- Función comunicativa.

Se distinguen cuatro grupos de elementos en el diseño de interfaces: elementos conceptuales, elementos visuales, elementos de relación y elementos prácticos.

Por lo tanto, la interfaz será un conjunto de elementos gráficos y un diseño de su distribución que permite una mejor presentación y navegación a través de la aplicación. Si no existen ambos factores unidos, si el resultado final de nuestro sitio es óptimo, será fruto de la casualidad. Podemos ver en las siguientes imágenes dos diseños, uno bueno y otro no tanto.



Figura 3. Ejemplos de un mal y un buen diseño gráfico

Como se puede ver en la figura 3, aparecen contrapuestas dos interfaces para una misma aplicación, en la izquierda podemos observar un “mal” diseño, se trata de una aplicación en la que los elementos no aparecen claramente diferenciados, no se puede leer bien el texto, ni se aprecian bien las imágenes, etc. Mientras que, en la aplicación de la derecha, la navegación del usuario resulta mucho más intuitiva, proporcionando un mayor grado de satisfacción, lo que se debe, entre otros factores, a que los elementos aparecen claramente dispuestos y no hay sobrecarga de estos.

En la construcción de cualquier aplicación y su interfaz correspondiente, debemos tener en cuenta diferentes fases, desde la definición de los objetivos que se persiguen con nuestro proyecto hasta el diseño visual resultante, pasando por las especificaciones funcionales, entre otros.

En la siguiente pila, vamos a definir todas estas fases que se deben tener presentes en el desarrollo de la interfaz.



Figura 4. Planos de jerarquía de diseño de una aplicación

Tras identificar las tareas que debe realizar una aplicación, en primer lugar definiremos los objetos y las acciones; este enfoque es similar a la definición de clases en un diseño orientado a objetos. Lo primero que se aconseja es realizar un modelado textual del escenario de la aplicación, distinguiendo entre sustantivos y verbos, los primeros defienden los objetos y elementos, y los segundos las acciones sobre los anteriores. Es habitual diferenciar los siguientes tipos de objetos:

- **Objetos origen:** elementos de la aplicación que permiten ser desplazados a cualquier otro punto, siendo depositados sobre un objeto destino.
- **Objetos destino:** elementos que reciben los objetos origen.
- **Objetos de la aplicación:** elementos propios de la aplicación que no permiten una interacción directa con el usuario.

Uno de los aspectos más importantes en cuanto al diseño de interfaces recae sobre el tipo de la pantalla donde será mostrado, debiendo analizar por completo el diseño gráfico que vamos a emplear, la colocación de los iconos, el dispositivo de interacción, los textos descriptivos, etc. Una vez finalizado el diseño, debemos realizar una revisión del mismo para descubrir los posibles problemas de funcionamiento y su corrección.

- **Tiempo de respuesta del sistema.** El tiempo de respuesta hace referencia a dos conceptos clave: duración y variabilidad. El primero queda definido como el tiempo que se emplea para completar una acción. Escoger este valor no es trivial, puesto que si es muy corto el usuario puede no haber completado la acción, pero si es demasiado largo, puede suponer su desistimiento.

- **Servicios de ayuda al usuario.** Existen dos tipos de ayuda al usuario: ayudas integradas en la aplicación, que aparecen en la mayoría de casos de forma automática, como sugerencia o guía al iniciar una tarea; y ayudas complementarias, que normalmente se encuentran en forma de manuales disponibles en la propia herramienta o a través de Internet. Estos manuales permiten filtrar el contenido que se busca a través de la generación de pequeñas consultas.
- **Etiquetado de órdenes.** Este aspecto se centra en el correcto diseño de la nomenclatura asociada a cada acción, es decir, que el elemento textual o visual correspondiente a una determinada función sea una palabra adecuada para cada acción, así como incluir atajos de teclado significativos y fáciles de recordar, o emplear teclas de función para las órdenes más usadas.

El primer paso para el diseño de una interfaz es la elaboración de una maqueta o prototipo. La consecución de una buena versión previa de lo que más adelante se va a desarrollar mejora la velocidad de desarrollo de la aplicación. Existen diferentes tipos de herramientas que nos permiten llevar a cabo una adecuada construcción de prototipos, incluyendo ventanas, menús, tratamiento de errores, cuadros de diálogo, etc.

Uno de los puntos más importantes para definir un buen diseño es la evaluación del mismo, el diseño de esta puede resultar determinante. Se aconseja al menos llevar a cabo el siguiente proceso:

1. A través de un formulario de carácter cuantitativo o cualitativo se recoge el grado de satisfacción del usuario. En el segundo de los casos será posible evaluar el tiempo empleado por el usuario para aprender el funcionamiento de la aplicación, el tiempo de lectura de la ayuda, etc.
2. Gracias a los datos recogidos, el diseñador puede comprobar si la interfaz cumple los requisitos planteados al comienzo del diseño.
3. Si el análisis anterior no cumple las expectativas de diseño, será necesario revisar de nuevo el mismo para que se adecue a las directrices iniciales, así como para solventar los errores o defectos que se hubieran podido detectar. Este proceso se repetirá hasta que se cumplan los requisitos previos de diseño para la interfaz.

3.3. Prototipos. Elementos clave de un prototipo

Un prototipo consiste en una maqueta o modelo de un diseño para que nos hagamos una idea del producto final que se obtendrá. El prototipo nos permite ver el resultado de distintos diseños finales, comprobar alguna funcionalidad o realizar test de usabilidad. De esta forma, ahorramos tiempo, esfuerzo o dinero, puesto que es más sencillo realizar cambios sobre un diseño previo que sobre un producto final.

La aplicación de prototipos al diseño de interfaces es fundamental, puesto que permite realizar multitud de diseños previos de los menús, elementos y demás partes del diseño para adecuarlos a las necesidades del cliente, antes de comenzar a escribir el código.

Alguna de las ventajas que justifica el uso del prototipo son:

- **Mejora la velocidad de desarrollo.** Es más eficiente realizar los cambios sobre un prototipo antes de comenzar su desarrollo que sobre un diseño definitivo, en el que aparezcan colores y tipografías, puesto que el cliente perderá la atención en el diseño base, que es el objeto principal del diseño de interfaces y del prototipo como primer paso de desarrollo.
- **Involucra al cliente.** Como se he comentado en el punto anterior, el cliente es el responsable de aprobar el diseño último del sitio, por lo tanto, será mucho más sencillo y útil proponer y hacer cambios sobre un esquema con poco detalle que sobre un diseño ya acabado. Es fácil involucrar al cliente en esta fase.

No existe un tipo único de prototipo, sino que encontramos múltiples, como esquemas de página, wireframes, prototipos, mockups, bocetos, sketches, diagramas,... Se pueden distinguir cuatro tipos:

- **Sketching:** Se trata de un boceto rápido, un esquema, para dibujar toda la interfaz de la aplicación, los procesos y las relaciones entre pantallas (a mano o digital). Este primer tipo es el que se suele emplear en la fase inicial, donde realizaremos diseños esquemáticos en papel. Se basa esencialmente en establecer la jerarquía de contenidos, pero sin detalles de diseño ni acabados complejos.
- **Wireframing:** se utiliza para dibujar con un cierto nivel de detalle las pantallas, los esbozos de contenido, las llamadas a la acción y, en general, la disposición física de los elementos (papel o digital). En esta fase se desarrolla y entrega una maqueta con base a lo “diseñado” en el paso previo. De esta forma es posible

validar los aspectos de diseño por parte del cliente. Lo fundamental de esta fase es la organización de los contenidos.

- **Mockups** son representaciones visuales estáticas más elaboradas y realistas de los diseños de las pantallas que muestran su aspecto real, incluyendo colores, tipografía e imágenes, pero no son interactivos. Permiten evaluar el aspecto visual y la experiencia antes de la implementación.
- **Prototipado:** para diseñar y ejecutar la interacción entre las pantallas que componen los procesos (solo digital). El prototipado se utiliza como paso final, puesto que permite evaluar no solo el diseño y la organización, sino también el funcionamiento, la interacción (menús, formularios, botones, iconos). Se emplea para hacer pruebas de usuarios antes de tener hecho el desarrollo completo e implantado del producto. Esto ahorra horas de desarrollo, ya que son necesarias menos versiones de la aplicación. Podemos encontrarnos tres clases de prototipos basados en su funcionalidad:
 - **Horizontal.** Modela muchas características de una aplicación, pero incorporando pocos detalles. Es el prototipo utilizado en las primeras etapas de diseño.
 - **Vertical.** Modela pocas características, pero en este caso se añaden más detalles.
 - **Diagonal.** Se trata de un prototipo mixto entre los dos anteriores. Hasta cierto nivel presenta las características del tipo horizontal, a partir del cual implementa las del tipo vertical.

El diseño del prototipo, en cualquiera de sus fases, se debe basar en los siguientes aspectos:

- Identificar los elementos que forman parte de cada una de las pantallas de una aplicación.
- Distribuir el número de elementos de la interfaz gráfica para que no exista saturación de elementos, pero haya suficiente información en la misma y la interacción sea correcta.
- Organización de la jerarquía de elementos, orden y disposición de los mismos.
- Extensión adecuada del diseño para aprovechar eficientemente el espacio en función del dispositivo.
- Patrones de diseño para estandarizar el de interfaces.
- Aspectos técnicos de usabilidad y accesibilidad.

Cuando hablamos de una interfaz gráfica, podemos diferenciar claramente dos áreas, aquella común a todas las ventanas de una misma aplicación y, por otro lado, la parte de contenido que varía de una ventana a otra.

La **jerarquía visual** es la disposición de los elementos. No se trata de algo trivial, sino que es necesario definir de forma eficiente las prioridades de comunicación, información e interacción.

Los elementos se deben situar de izquierda a derecha y de arriba hacia abajo para establecer una jerarquía que va de mayor a menor importancia. Lo más importante va arriba a la izquierda, y va perdiendo fuerza o importancia aquello que se relega hacia abajo y a la derecha.

3.4. Planteamiento y diseño de una interfaz

Para finalizar este primer tema de introducción, en el que ya hemos visto la importancia del diseño en las interfaces, así como algunas herramientas que nos pueden resultar útiles, nos centramos en los elementos clave que debemos tener en cuenta para crear el prototipo de nuestra interfaz.

- Los elementos que van a formar parte de nuestra aplicación. Debe haber un número suficiente, pero sin que haya saturación de estos.
- La extensión de la aplicación.
- Patrones de diseño que van a utilizarse para estandarizar el diseño de interfaces.
- Aspectos técnicos de usabilidad y accesibilidad.



Figura 5. Pasos para el diseño de una interfaz de una aplicación

Las áreas que habitualmente participan en el proyecto de diseño de una interfaz gráfica son: área de redacción, área de producción, área técnica y área artística.

3.4.1. Área de redacción

El área de redacción es la encargada de delimitar los pilares fundamentales de los que debe constar el proyecto que se está desarrollando. El aspecto principal que se debe establecer es el objeto final que busca nuestra aplicación. Por ejemplo, no es lo mismo realizar el diseño de una aplicación que se va a dedicar a vender repuestos de neumáticos, que una aplicación para la venta de entradas de conciertos.

3.4.2. Área de producción

Una vez que se ha definido el propósito final del proyecto, es necesario que este se encuentre en consonancia con las circunstancias del mercado actual, llevar a cabo un estudio de la viabilidad y trazar el plan de desarrollo más adecuado. Los agentes que participan en la producción son:

- **Cliente.** Se trata de quién encarga la creación y desarrollo de la aplicación del proyecto.
- **Usuario.** El público hacia el que va dirigida la aplicación. Son los receptores finales del proyecto. Por lo tanto, es interesante hacer un estudio de ellos, conocer sus necesidades y demandas, con el fin de mejorar el rendimiento de la aplicación.
- **Presupuesto.** Se trata del total económico que el cliente desea destinar a la construcción de la aplicación.
- **Plan de trabajo.** Calendario de entregas, distribución de las tareas, etc.

3.4.3. Área técnica

Esta área se encuentra constituida por los responsables técnicos, que son los encargados de realizar un estudio de los requisitos del proyecto relativos a su programación, por ejemplo, en HTML, CSS y bases de datos. Hasta aquí el resto de las áreas se han basado en el de *sketching*, en un primer momento, y en *wireframing*. El área técnica comienza el desarrollo de los primeros prototipos y maquetas. Recordemos que estos ya comienzan a incorporar los primeros elementos de interacción, hasta llegar al producto final.

3.4.4. Área artística

Esta última área se centra en la estética final del proyecto, en base al propósito final de uso de la aplicación, así como a los informes de usuario y los requisitos del cliente. Hay que tener en cuenta que el estilo variará en función de la aplicación y de los usuarios. El estilo del proyecto se debe convertir en una seña de identidad ya que en esto consiste realizar un buen diseño gráfico, en dotar de personalidad propia a una aplicación y que esta sea fácilmente reconocible en cualquier situación.

3.5. El color

El ojo humano solo es capaz de percibir los denominados **colores aditivos**; a través de la combinación de estos le es posible obtener el resto de los colores. Estos son: azul (B), rojo (R) y verde (G).

En este apartado analizaremos en qué consiste el sistema de representación RGB, así como las principales propiedades del color, que modifican y redefinen el sistema de color base.

3.5.1. Sistema RGB

Un ordenador será capaz de obtener la representación de todos los colores utilizando el sistema **RGB**, o lo que es lo mismo, el sistema Red-Green-Blue. Indicando la proporción de cada uno de ellos dentro de la combinación de estos tres, dará lugar a toda la paleta de colores conocida.

Para representar cada color de forma que pueda ser traducido por el ordenador se utilizan 8 bits para codificar cada uno de los colores aditivos, es decir, se establece la proporción de cada color que va a formar parte de la combinación de tres. La escala monocromática de un color tendrá 256 (2^8) valores.

A la hora de representar cada uno de los colores, es posible utilizar tanto el sistema de numeración decimal (0 a 255) como el hexadecimal, donde cada uno de los dígitos se codifica con 8 bits binarios que, agrupados en bloques de 4 bits, nos devuelve el valor correspondiente en hexadecimal.

El número de combinaciones de colores se calcula multiplicando el número máximo de grados en la escala monocromática de cada color, $256 \times 256 \times 256$, lo que nos da 16.777.216 colores.

Ejemplo

El color amarillo está formado por:

Rojo = 255 → 1111 1111 → ff

Verde = 255 → 1111 1111 → ff

Azul = 0 → 0000 0000 → 00

En hexadecimal queda expresado por: #ffff00

3.5.2. Matiz, saturación y brillo

Además del grado en la escala monocromática de cada uno de los colores del sistema RGB, los colores presentan tres propiedades que permiten distinguirse unos de otros, estas son: el matiz, la saturación y el brillo.

El modelo **HSL** (Hue, Saturation, Lightness) describe el color en términos de tono o matiz, saturación y luminosidad o brillo y es útil para seleccionar colores armónicos. Las propiedades de matiz, saturación y brillo nos permiten definir los colores como cromáticos, complementarios o cercanos, así como definir el contraste de color.

- **Matiz.** Atributo que permite distinguir un color de otro. Los tres matices primarios son los colores aditivos, verde, rojo y azul; el resto de colores se obtiene mezclando estos tres. El matiz permite definir dos colores como complementarios cuando está uno frente al otro en el círculo cromático.



Figura 6. Círculo cromático de matices

- **Saturación.** Este atributo define la intensidad de un color. Puede relacionarse con el ancho de banda de luz que estamos visualizando, por lo tanto, queda condicionado por el nivel de gris presente en un color: cuanto mayor sea el nivel de gris, menos saturado será un color, y será menos intenso.



Figura 7. Escala de color con matiz rojo modificando sus valores de saturación

- **Brillo.** Atributo que define la cantidad de luz de un color. Representa lo oscuro (si se le añade negro) o claro (si se le añade blanco) que un color respecto de su patrón, es decir, respecto del color puro sin modificar el brillo. En una composición de colores en diseño gráfico, cuanto más brillante sea un color, más cerca parece estar.



Figura 8. Escala de color con matiz rojo modificando sus valores de brillo

En el diseño de interfaces gráficas, la selección adecuada de la carta de colores es muy importante, puesto que esto puede condicionar la experiencia de navegación del usuario, determinándola por completo. La opción más sencilla es escoger la **monocromía**, que consiste en elegir un solo color del círculo cromático y, a continuación, obtenemos su variedad de tonalidades, añadiendo blanco o negro.

Otra de las alternativas más utilizadas son los denominados **colores vecinos**. Estos son armónicos, que proporcionan estabilidad en el diseño de colores. Se denominan *vecinos* aquellos colores que se encuentran en un rango de 90° en el círculo cromático.

3.5.3. Psicología del color

Cada color puede transmitir diferentes emociones y significados, por ejemplo:

- **Rojo:** energía, urgencia, alerta.
- **Azul:** confianza, calma, profesionalidad.
- **Verde:** naturaleza, crecimiento, seguridad.
- **Amarillo:** atención, optimismo, advertencia.

El diseño debe considerar estos aspectos para alinear la interfaz con el mensaje y el público objetivo.

Además hay otras características del color a tener en cuenta:

- **Contraste:** Importante para que el texto y los elementos sean legibles y destacables, especialmente para personas con dificultades visuales.
- **Armonía:** Combinación de colores que resultan agradables y coherentes.
- **Accesibilidad:** Garantizar que combinaciones de color funcionen para usuarios con daltonismo u otras discapacidades visuales.

3.6. Herramientas de diseño y prototipado (sin código).

En el diseño de interfaces, las herramientas de prototipado permiten crear representaciones visuales y funcionales de la interfaz sin necesidad de programar, facilitando la comunicación, iteración y validación antes del desarrollo.

Algunas de las más populares son:

- **Figma:** Herramienta online colaborativa que permite diseñar interfaces, crear

prototipos interactivos y trabajar en equipo en tiempo real. Es muy usada por su facilidad y capacidades avanzadas.

- **Adobe XD:** Software de Adobe orientado al diseño y prototipado de interfaces con herramientas intuitivas, integración con el ecosistema Adobe y opciones para compartir prototipos.
- **Sketch:** Aplicación para macOS enfocada en diseño UI/UX, conocida por su simplicidad, gran cantidad de plugins y uso profesional en diseño web y móvil.
- **Pencil Project:** Herramienta de código abierto para crear diagramas y wireframes, ideal para entornos educativos o quienes buscan software gratuito y sencillo.
- **Balsamiq:** Software que simula el estilo de boceto a mano para wireframes rápidos y comprensibles, favoreciendo la concentración en la estructura y usabilidad antes del diseño visual.

En la siguiente tabla se observa una comparativa básica de las herramientas anteriores.

Herramienta	Plataformas	Nivel de dificultad	Colaboración	Precio
Figma	Web, Windows, Mac	Bajo - Medio	Sí	Gratis y suscripción
Adobe XD	Windows, Mac	Medio	Sí	Gratis y suscripción
Sketch	Mac	Medio	Limitada	Pago
Pencil Project	Windows, Mac, Linux	Bajo	No	Gratis
Balsamiq	Web, Windows, Mac	Bajo	Limitada	Pago

Con estas herramientas se pueden:

- **Crear wireframes:** Diseño básico de la estructura, disposición y funcionalidad sin distracciones visuales.
- **Desarrollar mockups:** Maquetas visuales detalladas que representan el aspecto final, incluyendo colores, tipografías, iconos y estilos.
- **Prototipar interacciones:** Definir flujos y transiciones para simular la experiencia de usuario y validar la navegación.